
STOCHASTIC INVERSE REINFORCEMENT LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

The goal of the inverse reinforcement learning (IRL) problem is to recover the reward functions from expert demonstrations. However, the IRL problem like any ill-posed inverse problem suffers the congenital defect that the policy may be optimal for many reward functions, and expert demonstrations may be optimal for many policies. In this work, we generalize the IRL problem to a well-posed expectation optimization problem *stochastic inverse reinforcement learning* (SIRL) to recover the probability distribution over reward functions. We adopt the Monte Carlo expectation-maximization (MCEM) method to estimate the parameter of the probability distribution as the first solution to the SIRL problem. The solution is *succinct, robust, and transferable* for a learning task and can generate alternative solutions to the IRL problem. Through our formulation, it is possible to observe the intrinsic property for the IRL problem from a *global* viewpoint, and our approach achieves a considerable performance on the *objectworld*.

1 INTRODUCTION

The IRL problem addresses an inverse problem that a set of expert demonstrations determines a reward function over a Markov decision process (MDP) if the model dynamics are known [Russell \(1998\)](#); [Ng et al. \(2000\)](#). The recovered reward function provides a *succinct, robust, and transferable* definition of the learning task and completely determines the optimal policy. However, the IRL problem is ill-posed that the policy may be optimal for many reward functions and expert demonstrations may be optimal for many policies. For example, all policies are optimal for a constant reward function. In a real-world scenario, experts always act sub-optimally or inconsistently, which is another challenge.

To overcome these limitations, two classes of probabilistic approaches for the IRL problem are proposed, i.e., Bayesian inverse reinforcement learning (BIRL) [Ramachandran & Amir \(2007\)](#) based on Bayesians' maximum a posteriori (MAP) estimation and maximum entropy IRL (MaxEnt) [Ziebart et al. \(2008\)](#); [Ziebart \(2010\)](#) based on frequentists' maximum likelihood (MLE) estimation. BIRL solves for the distribution of reward functions without an assumption that experts behave optimally, and encode the external *a priori* information in a choice of a prior distribution. However, BIRL also suffers from the practical limitation that a large number of algorithmic iterations is required for the procedure of Markov chain Monte Carlo (MCMC) in a sampling of posterior over reward functions. Advanced techniques, for example Kernel technique [Michini & How \(2012\)](#) and gradient method [Choi & Kim \(2011\)](#), are proposed to improve the efficiency and tractability of this situation.

MaxEnt employs the principle of maximum entropy to resolve the ambiguity in choosing demonstrations over a policy. This class of methods, inheriting the merits from previous non-probabilistic IRL approaches including [Ng et al. \(2000\)](#); [Abbeel & Ng \(2004\)](#); [Ratliff et al. \(2006\)](#); [Abbeel et al. \(2008\)](#); [Syed & Schapire \(2008\)](#); [Ho et al. \(2016\)](#), imposes regular structures of reward functions in a combination of hand-selected features. Formally, the reward function is a linear or nonlinear combination of the feature basis functions which consists of a set of real-valued functions $\{\phi_i(s, a)\}_{i=1}$ hand-selected by experts. The goal of this approach is to find the best-fitting weights of feature basis functions through the MLE approach. [Wulfmeier et al. \(2015\)](#) and [Levine et al. \(2011\)](#) use deep neural networks and Gaussian processes to fit the parameters based on demonstrations respectively but still suffer from the problem that the true reward shaped by the changing environment dynamics.

Influenced by the work of [Finn et al. \(2016a;b\)](#), [Fu et al. \(2017\)](#) propose a framework called adversarial IRL (AIRL) to recover robust reward functions in a changing dynamics based on adversarial learning and achieves superior results. Compared with AIRL, another adversarial method called generative

adversarial imitation learning (GAIL) [Ho & Ermon \(2016\)](#) seeks to directly recover the expert’s policy rather than reward functions. Many follow-up methods enhance and extend GAIL for multipurpose in various application scenarios [Li et al. \(2017\)](#); [Hausman et al. \(2017\)](#); [Wang et al. \(2017\)](#). However, GAIL is in a lack of an explanation of expert’s behavior and a portable representation for the knowledge transfer which are the merits of the class of the MaxEnt approach, because the MaxEnt approach is equipped with the "transferable" regular structures over reward functions.

In this paper, under the framework of the MaxEnt approach, we propose a generalized perspective of studying the IRL problem called *stochastic inverse reinforcement learning* (SIRL). It is formulated as an expectation optimization problem aiming to recover a probability distribution over the reward function from expert demonstrations. The solution of SIRL is *succinct* and *robust* for the learning task in the meaning that it can generate more than one weight over feature basis functions which compose alternative solutions to the IRL problem. Benefits of the class of the MaxEnt method, the solution to our generalized problem SIRL is also *transferable*. Since of the intractable integration in our formulation, we employ the Monte Carlo expectation-maximization (MCEM) approach [Wei & Tanner \(1990\)](#) to give the first solution to the SIRL problem in a model-based environment. In general, the solutions to the IRL problem are not always best-fitting in the previous approaches because a highly nonlinear inverse problem with the limited information is very likely to get trapped in a secondary maximum in the recovery. Taking advantage of the Monte Carlo mechanism of a *global* exhaustive search, our MCEM approach avoids the secondary maximum and theoretically convergent demonstrated by pieces of literature [Caffo et al. \(2005\)](#); [Chan & Ledolter \(1995\)](#). Our approach is also quickly convergent because of the preset simple geometric configuration over weight space in which we approximate it with a Gaussian Mixture Model (GMM). Hence, our approach works well in a real-world scenario with a small and variability set of expert demonstrations.

In particular, the contributions of this paper are threefold:

1. We generalize the IRL problem to a well-posed expectation optimization problem SIRL.
2. We provide the first theoretically existing solution to SIRL by the MCEM approach.
3. We show the effectiveness of our approach by comparing the performance of the proposed method to those of the previous algorithms on the *objectworld*.

2 PRELIMINARY

An MDP is a tuple $\mathcal{M} := \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, and the transition function (a.k.a. model dynamics) $\mathcal{T} := \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$ for $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}$ is the probability of being current state s , taking action a and yielding next state s' . Reward function $\mathcal{R}(s, a)$ is a real-valued function and $\gamma \in [0, 1)$ is the discount factor. A policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ is deterministic or stochastic, where the deterministic one is written as $a = \pi(s)$, and the stochastic one is as a conditional distribution $\pi(a|s)$. Sequential decisions are recorded in a series of episodes which consist of states s , actions a , and rewards r . The goal of reinforcement learning aims to get optimal policy π^* for maximizing the expected total reward, i.e.

$$\pi^* := \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \cdot \mathcal{R}(s_t, a_t) \middle| \pi \right].$$

Given an MDP without a reward function $\mathcal{R}$, i.e., $\text{MDP} \setminus \mathcal{R} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T} \rangle$, and m expert demonstrations $\zeta^E := \{\zeta^1, \dots, \zeta^m\}$. Each expert demonstration ζ^i is a sequential of state-action pairs. The goal of the IRL problem is to estimate the unknown reward function $\mathcal{R}(s, a)$ from expert demonstrations ζ^E and the reward functions compose a complete MDP. The estimated complete MDP yields an optimal policy that acts as closely as the expert demonstrations.

2.1 REGULAR STRUCTURE OF REWARD FUNCTIONS

In this section, we provide a formal definition of the regular (linear/nonlinear) structure of reward functions. The linear structure [Ng et al. \(2000\)](#); [Ziebart et al. \(2008\)](#); [Syed & Schapire \(2008\)](#); [Ho](#)

et al. (2016) is an M linear combination of feature basis functions, written as,

$$\mathcal{R}(s, a) := \sum_i^M \alpha_i \cdot \phi_i(s, a),$$

where $\phi_i : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}^d$ are a d -dimensional feature functions hand-selected by experts.

The nonlinear structure Wulfmeier et al. (2015) is of the form as follows,

$$\mathcal{R}(s, a) := \mathcal{N}(\phi_1(s, a), \dots, \phi_M(s, a)),$$

where $\mathcal{N}$ is neural networks of hand-crafted reward feature basis functions $\{\phi_i(s, a)\}_{i=1}^M$.

In the following section, we propose an optimization problem whose solution is a probability distribution over weights of basis functions $\{\phi_i(s, a)\}_{i=1}^M$.

2.2 PROBLEM STATEMENT

Given an MDP $\mathcal{R} := (\mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma)$ with a known transition function $\mathcal{T} := \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$ for $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}$ and a hand-crafted reward feature basis function $\{\phi_i(s)\}_{i=1}^M$. A stochastic regular structure on the reward function assumes weights $\mathcal{W}$ of the reward feature functions $\phi_i(s)$, which are random variables with a reward conditional probability distribution $\mathcal{D}^{\mathcal{R}}(\mathcal{W} | \zeta^E)$ conditional on expert demonstrations ζ^E . Parametrizing $\mathcal{D}^{\mathcal{R}}(\mathcal{W} | \zeta^E)$ with parameter Θ , our aim is to estimate the best-fitting parameter Θ^* from the expert demonstrations ζ^E , such that $\mathcal{D}^{\mathcal{R}}(\mathcal{W} | \zeta^E, \Theta^*)$ more likely generates weights to compose reward functions as the ones derived from expert demonstrations, which is called *stochastic inverse reinforcement learning* problem.

Suppose a representative trajectory class $\mathcal{C}_\epsilon^E$ satisfies that each trajectory element set $\mathcal{O} \in \mathcal{C}_\epsilon^E$ is a subset of expert demonstrations ζ^E with the cardinality at least $\epsilon \cdot |\zeta^E|$, where ϵ is a preset threshold and $|\zeta^E|$ is the number of expert demonstrations, written as,

$$\mathcal{C}_\epsilon^E := \{\mathcal{O} | \mathcal{O} \subset \zeta^E \text{ with } |\mathcal{O}| \geq \epsilon \cdot |\zeta^E|\}.$$

Integrate out unobserved weights $\mathcal{W}$, and the SIRL problem is formulated to estimate parameter Θ on an expectation optimization problem over the representative trajectory class as follows:

$$\Theta^* := \arg \max_{\Theta} \mathbb{E}_{\mathcal{O} \in \mathcal{C}_\epsilon^E} \left[\int_{\mathcal{W}} f_{\mathcal{M}}(\mathcal{O}, \mathcal{W} | \Theta) d\mathcal{W} \right], \quad (1)$$

where trajectory element set $\mathcal{O}$ assumes to be uniformly distributed for the sake of simplicity in this study, and $f_{\mathcal{M}}$ is the conditional joint probability density function of trajectory element $\mathcal{O}$ and weights $\mathcal{W}$ for reward feature functions conditional on parameter Θ .

2.2.1 NOTE:

- Problem 1 is well-posed and typically not intractable analytically.
- Trajectory element set $\mathcal{O}$ is usually known from the rough estimation of the statistics in expert demonstrations in practice.
- We introduce *representative trajectory class* to overcome the limitation of the sub-optimality of expert demonstrations ζ^E Ramachandran & Amir (2007), which is also called a lack of sampling representativeness in statistics Kruskal & Mosteller (1979), e.g., driver's demonstrations encode his own preferences in driving style but may not reflect the true rewards of an environment. With the technique of representative trajectory class, each subset of driver's demonstrations (i.e. trajectory element set $\mathcal{O}$) constitutes a subproblem in Problem 1.

3 METHODOLOGY

In this section, we propose a novel approach to estimate the best-fitting parameter Θ^* in Problem 1, which is called the two-stage hierarchical method, a *variant of MCEM* method.

3.1 TWO-STAGE HIERARCHICAL METHOD

The two-stage hierarchical method requires us to write parameter Θ in a profile form $\Theta := (\Theta_1, \Theta_2)$. The conditional joint density $f_{\mathcal{M}}(\mathcal{O}, \mathcal{W}|\Theta)$ in Equation 1 can be written as the product of two conditional densities $g_{\mathcal{M}}$ and $h_{\mathcal{M}}$ as follows:

$$f_{\mathcal{M}}(\mathcal{O}, \mathcal{W}|\Theta_1, \Theta_2) = g_{\mathcal{M}}(\mathcal{O}|\mathcal{W}, \Theta_1) \cdot h_{\mathcal{M}}(\mathcal{W}|\Theta_2). \quad (2)$$

Take the log of both sides in Equation 2, and we have

$$\log f_{\mathcal{M}}(\mathcal{O}, \mathcal{W}|\Theta_1, \Theta_2) = \log g_{\mathcal{M}}(\mathcal{O}|\mathcal{W}, \Theta_1) + \log h_{\mathcal{M}}(\mathcal{W}|\Theta_2). \quad (3)$$

We maximize the right side of Equation 3 over the profile parameter Θ in the expectation-maximization (EM) update steps at the t -th iteration independently as follows,

$$\Theta_1^{t+1} := \arg \max_{\Theta_1} \mathbb{E} \left(\log g_{\mathcal{M}}(\mathcal{O}|\mathcal{W}, \Theta_1) \middle| \mathcal{C}_{\epsilon}^E, \Theta^t \right); \quad (4)$$

$$\Theta_2^{t+1} := \arg \max_{\Theta_2} \mathbb{E} \left(\log h_{\mathcal{M}}(\mathcal{W}|\Theta_2) \middle| \Theta^t \right). \quad (5)$$

3.1.1 INITIALIZATION

We randomly initialize profile parameter $\Theta^0 := (\Theta_1^0, \Theta_2^0)$ and sample a collection of N_0 rewards weights $\{\mathcal{W}_1^{\Theta^0}, \dots, \mathcal{W}_{N_0}^{\Theta^0}\} \sim \mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta_2^0)$. The reward weights $\mathcal{W}_i^{\Theta^0}$ compose reward $R_{\mathcal{W}_i^{\Theta^0}}$ in each learning task $\mathcal{M}_i^0 := (\mathcal{S}, \mathcal{A}, \mathcal{T}, R_{\mathcal{W}_i^{\Theta^0}}, \gamma)$ for $i = 1, \dots, N_0$.

3.1.2 FIRST STAGE

In the first stage, we aim to update parameter Θ_1 in the intractable expectation of Equation 4. Specifically, we take a Monte Carlo method to estimate model parameters Θ_1^{t+1} in an empirical expectation at the t -th iteration,

$$\mathcal{E}[\log g_{\mathcal{M}}(\mathcal{O}|\mathcal{W}, \Theta_1^{t+1}) \middle| \mathcal{C}_{\epsilon}^E, \Theta^t] := \frac{1}{N_t} \cdot \sum_{i=1}^{N_t} \log g_{\mathcal{M}_i^t}(\mathcal{O}_i^t | \mathcal{W}_i^{\Theta^t}, \Theta_1^{t+1}), \quad (6)$$

where reward weights at the t -th iteration $\mathcal{W}_i^{\Theta^t}$ are randomly drawn from the reward conditional probability distribution $\mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta^t)$ and compose a set of learning tasks $\mathcal{M}_i^t := (\mathcal{S}, \mathcal{A}, \mathcal{T}, R_{\mathcal{W}_i^{\Theta^t}}, \gamma)$ with a trajectory element set $\mathcal{O}_i^t$ uniformly drawn from representative trajectory class $\mathcal{C}_{\epsilon}^E$, for $i = 1, \dots, N_t$.

The parameter Θ_1^{t+1} in Equation 6 has N_t coordinates written as $\Theta_1^{t+1} := ((\Theta_1^{t+1})_1, \dots, (\Theta_1^{t+1})_{N_t})$. For each learning task $\mathcal{M}_i^t$, the i -th coordinate $(\Theta_1^{t+1})_i$ is derived from maximization of a posteriori,

$$(\Theta_1^{t+1})_i := \arg \max_{\theta} \log g_{\mathcal{M}_i^t}(\mathcal{O}_i^t | \mathcal{W}_i^{\Theta^t}, \theta),$$

which is a convex formulation maximized by a gradient ascent method.

In practice, we move m steps uphill to the optimum in each learning task $\mathcal{M}_i^t$. The update formula of m -step reward weights $\mathcal{W}_i^{m\Theta^t}$ is written as

$$\mathcal{W}_i^{m\Theta^t} := \mathcal{W}_i^{\Theta^t} + \sum_{i=1}^m \lambda_i^t \cdot \nabla_{(\Theta_1)_i} \log g_{\mathcal{M}_i^t}(\mathcal{O}_i^t | \mathcal{W}_i^{\Theta^t}, (\Theta_1)_i),$$

where the learning rate λ_i^t at the t -th iteration is preset. Hence, the parameter Θ_1^{t+1} is represented as $\Theta_1^{t+1} := (\mathcal{W}_1^{m\Theta^t}, \dots, \mathcal{W}_{N_t}^{m\Theta^t})$.

3.1.3 SECOND STAGE

In the second stage, we aim to update parameter Θ_2 in the intractable expectation of Equation 5. Specifically, we consider the empirical expectation at the t -th iteration as follows,

$$\mathcal{E}(\log h_{\mathcal{M}}(\mathcal{W}|\Theta_2^{t+1})|\Theta^t) := \frac{1}{N_t} \cdot \sum_{i=1}^{N_t} \log h_{\mathcal{M}_i^t}(\mathcal{W}_i^{m\Theta^t}|\Theta_2^{t+1}), \quad (7)$$

where $h_{\mathcal{M}}$ is implicit but fitting a set of m -step reward weights $\{\mathcal{W}_i^{m\Theta^t}\}_{i=1}^{N_t}$ in a generative model yields a large empirical expectation value. The reward conditional probability distribution $\mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta_2^{t+1})$ is a generative model formulated as a Gaussian Mixture Model (GMM), i.e.

$$\mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta_2^{t+1}) := \sum_{k=1}^K \alpha_k \cdot \mathcal{N}(\mathcal{W}|\mu_k, \Sigma_k),$$

where $\alpha_k \geq 0$ and $\sum_{k=1}^K \alpha_k = 1$, and parameter set $\Theta_2^{t+1} := \{\alpha_k; \mu_k, \Sigma_k\}_{k=1}^K$.

We estimate parameter Θ_2^{t+1} in GMM by EM approach and initialize GMM with the t -th iteration parameter Θ_2^t with the procedure as follows:

For $i = 1, \dots, N_t$, we have

- Expectation Step: Compute responsibility γ_{ij} for m -step reward weight $\mathcal{W}_i^{m\Theta^t}$,

$$\gamma_{ij} := \frac{\alpha_j \cdot \mathcal{N}(\mathcal{W}_i^{m\Theta^t}|\mu_j, \Sigma_j)}{\sum_{k=1}^K \alpha_k \cdot \mathcal{N}(\mathcal{W}_i^{m\Theta^t}|\mu_k, \Sigma_k)}.$$

- Maximization Step: Compute weighted mean μ_j and variance Σ_j by,

$$\mu_j := \frac{\sum_{i=1}^{N_t} \gamma_{ij} \cdot \mathcal{W}_i^{m\Theta^t}}{\sum_{i=1}^{N_t} \gamma_{ij}}; \quad \alpha_j := \frac{1}{N_t} \cdot \sum_{i=1}^{N_t} \gamma_{ij}; \quad \Sigma_j := \frac{\sum_{i=1}^{N_t} \gamma_{ij} \cdot (\mathcal{W}_i^{m\Theta^t} - \mu_j) \cdot (\mathcal{W}_i^{m\Theta^t} - \mu_j)^T}{\sum_{i=1}^{N_t} \gamma_{ij}}.$$

After EM converges, $\Theta_2^{t+1} := \{\alpha_k; \mu_k, \Sigma_k\}_{k=1}^K$ and profile parameter $\Theta^{t+1} := (\Theta_1^{t+1}, \Theta_2^{t+1})$.

Finally, when the two-stage hierarchical method converges, parameter Θ_2 of profile parameter Θ is our desired best-fitting parameter Θ^* for $\mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta^*)$.

3.2 TERMINATION CRITERIA

In this section, we discuss the termination criteria in our algorithm. EM terminates usually when the parameters do not substantively change after enough iterations. For example, one classic termination criterion in EM terminates at the t -th iteration satisfying as follows,

$$\max \frac{|\theta^t - \theta^{t-1}|}{|\theta^t| + \delta_{EM}} < \epsilon_{EM},$$

for user-specified δ_{EM} and ϵ_{EM} , where θ is the model parameter in EM.

However, the same termination criterion for MCEM has a risk of early terminating because of the Monte Carlo error in the update step. Hence, we adopt a practical method in which the following stopping criterion holds in three consecutive times,

$$\max \frac{|\Theta^t - \Theta^{t-1}|}{|\Theta^t| + \delta_{MCEM}} < \epsilon_{MCEM},$$

for user-specified δ_{MCEM} and ϵ_{MCEM} [Booth & Hobert \(1999\)](#). Other stopping criteria for MCEM refers to [Caffo et al. \(2005\)](#); [Chan & Ledolter \(1995\)](#).

3.3 CONVERGENCE ISSUE

The convergence issue of MCEM is more complicated than EM. In light of model-based interactive MDP\mathcal{R}, we can always increase the sample size during each iteration of MCEM. In practice, we require the Monte Carlo sample size satisfy the following inequality,

$$\sum_t \frac{1}{N_t} < \infty.$$

Additional requirement for the convergence property is a compact assumption over that parameter space. A comprehensive proof refers to [Chan & Ledolter \(1995\)](#); [Fort et al. \(2003\)](#).

The pseudocode is given in Appendix.

4 EXPERIMENTS

We evaluate our approach on a classic environment *objectworld* introduced by [Levine et al. \(2011\)](#) which is a particularly challenging environment with a large number of irrelevant features and the highly nonlinearity of the reward functions. Note that since almost only *objectworld* provides a tool that allows analysis and display the evolution procedure of the SIRL problem in a 2D heat map, we skip the typical invisible physics-based control tasks for the evaluation of our approach, i.e. cartpole [Barto et al. \(1983\)](#), mountain car [Moore \(1990\)](#), MuJoCo [Todorov et al. \(2012\)](#), and etc.

We employ the expected value difference (EVD) proposed by [Levine et al. \(2011\)](#) to be the metric of optimality as follows:

$$\text{EVD}(\mathcal{W}) := \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \cdot R(s_t, a_t) \middle| \pi^*\right] - \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \cdot R(s_t, a_t) \middle| \pi(\mathcal{W})\right],$$

which is a measure of the difference between the expected reward earned under the optimal policy π^* , given by the true reward, and the policy derived from the rewards sampling from our reward conditional probability distribution $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$, where Θ^* is the best estimation parameter in our approach.

4.1 OBJECTWORLD

The *objectworld* is a learning environment for the IRL problem. It is an $N \times N$ grid board with colored objects placed in randomly selected cells. Each colored object is assigned one inner color and one outer color from C preselected colors. Each cell on the grid board is a state, and stepping to four neighbor cells (up, down, left, right) or staying in place (stay) are five actions with a 30% chance of moving in a random direction.

The ground truth of the reward function is defined in the following way. Suppose two primary colors of C preselected colors are red and blue. The reward of a state is 1 if the state is within 3 steps of an outer red object and 2 steps of an outer blue object, -1 if the state is within 3 steps of an outer red object, and 0 otherwise. The other pairs of inner and outer colors are distractors. Continuous and discrete versions of feature basis functions are provided. For the continuous version, $\phi(s)$ is a $2C$ -dimensional real-valued feature vector. Each dimension records the Euclidean distance from the state to objects. For example, the first and second coordinates are the distances to the nearest inner and outer red object respectively, and so on through all C colors. For the discrete version, $\phi(s)$ is a $(2C \cdot N)$ -dimensional binary feature vector. Each N -dimensional vector records a binary representation of the distance to the nearest inner or outer color object with the d -th coordinate 1 if the corresponding continuous distance is less than d .

4.2 EVALUATION PROCEDURE AND ANALYSIS

In this section, we design three tasks to evaluate the effectiveness of our generative model *reward conditional probability distribution* $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$. For each task, the environment setting is as follows. The instance of a 10×10 *objectworld* has 25 random objects with 2 colors and a 0.9 discount factor. 20 expert demonstrations are generated according to the given optimal policy for the

recovery. The length of each expert demonstration is 5-grid size trajectory length. Four algorithms for the evaluation includes MaxEnt, DeepMaxEnt, SIRL, and DSIRL, where SIRL and DSIRL are implemented as Algorithm 2 in Appendix. The weights are drawn from reward conditional probability distribution $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$ as the coefficients of feature basis functions $\{\phi_i(s, a)\}_{i=1}$.

In our evaluation, SIRL and DSIRL start from 10 samples and double the sample size per iteration until it converges for the convergence issue, refer to Section 3.2. In the first stage, the epochs of algorithm iteration are set to 20 and the learning rates are 0.01. The parameter ϵ in representative trajectory set $\mathcal{O}_\epsilon^E$ is preset as 0.95. In the second stage, 3-component GMM for SIRL and DSIRL is set with at most 1000 iterations before convergence. Additionally, the architecture of neural networks in DeepMaxEnt and DSIRL are implemented as 3-layer fully-connected with the sigmoid function.

4.2.1 EVALUATION PLATFORM

All the methods are implemented in Python 3.5 and Theano 1.0.0 with a machine learning distributed framework *Ray* Moritz et al. (2018). The experiments are conducted on a machine with Intel(R) Core(TM) i7-7700 CPU @ 3.60GHz and Nvidia GeForce GTX 1070 GPU.

4.2.2 RECOVERY EXPERIMENT

In the recovery experiment, we compare the true reward function, the optimal policy, and the optimal value with ones derived from expert demonstrations under the four methods. Since our approach is an average of all the outcomes which are prone to imitate the true optimal value from expert demonstrations, we use the mean of reward conditional probability distribution for SIRL and DSIRL as a comparison.

In Figure 1, the EVD of the optimal values in the last row are 48.9, 31.1, 33.7 and 11.3 for four methods respectively, and the covariances of GMM model for SIRL and DSIRL are limited up to 5.53 and 1.36 on each coordinate respectively. It yields that in a highly nonlinear inverse problem, the recovery abilities of SIRL and DSIRL are better than MaxEnt’s and DeepMaxEnt’s respectively. The reason is mainly because Monte Carlo mechanism in our approach alleviates the problem of getting stuck in local minima by allowing random exit from it.

4.2.3 ROBUSTNESS EXPERIMENT

In the robustness experiment, we evaluate the robustness of our approach that solutions generated by $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$ are effective to the IRL problem. To capture the robust solutions, we design the following generative algorithm with the pseudocode in Algorithm 1.

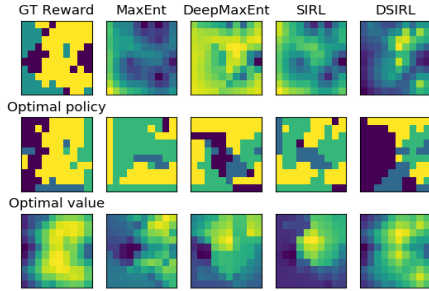


Figure 1: Results for recovery experiment.

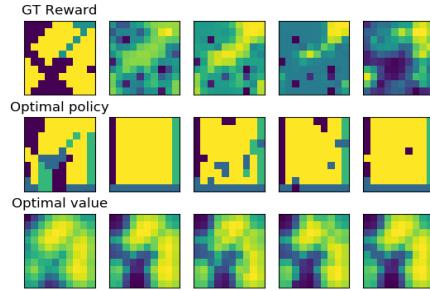


Figure 2: Results for the robustness experiment.

In the right generative algorithm, we use Frobenius norm to measure the distance between weights drawn from $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$ as follows,

$$\|\mathcal{W}\|_{\mathcal{F}} := \sqrt{\text{Tr}(\mathcal{W} \cdot \mathcal{W}^T)}.$$

We also constrain that each drawn weight $\mathcal{W} \sim \mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$ in the solution set $\mathcal{G}$ satisfies as follows,

$$\|\mathcal{W} - \mathcal{W}'\|_{\mathcal{F}} > \delta \text{ and } \text{EVD}(\mathcal{W}) < \epsilon,$$

where $\mathcal{W}'$ represents any member in the solution set $\mathcal{G}$. δ, ϵ are the preset thresholds in the generative algorithm.

Algorithm 1: Generative Algorithm

Input: $\mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$, required solution set size N , and preset thresholds ϵ and δ .

Output: Solution set $\mathcal{G} := \{\mathcal{W}_i\}_{i=1}^N$.

```

while  $i < N$  do
   $\mathcal{W} \sim \mathcal{D}(\mathcal{W}|\zeta^E, \Theta^*)$ ;
  for any  $\mathcal{W}' \in \mathcal{S}$  do
    if  $\|\mathcal{W} - \mathcal{W}'\|_{\mathcal{F}} > \delta$  and  $\text{EVD}(\mathcal{W}) < \epsilon$  then
       $\mathcal{G} \leftarrow \mathcal{W}$ ;
    end
  end
   $i \leftarrow i + 1$ ;
end

```

In Figure 2, the right column figures are generated from weights in the solution set $\mathcal{G}$ whose EVD values are around 10.2. Note that the recovered reward function in the first row has a similar but different pattern appearance. The optimal value derived from these recovered reward functions has a very small EVD value with the true reward. It yields the effectiveness of our *robust* generative model which can generate more than one solutions to the IRL problem.

4.2.4 HYPERPARAMETER EXPERIMENT

In the hyperparameter experiment, we evaluate the effectiveness of our approach under the influence of different preset quantities and qualities of expert demonstrations. The amount of information carried in expert demonstrations composes a specific learning environment, and hence it has an impact on the effectiveness of our generative model. We verify three hyperparameters including the number of expert demonstrations in Figure 3, the trajectory length of expert demonstrations in Figure 4 and the portion size in representative trajectory class $\mathcal{C}_\epsilon^E$ in Figure 5 on the *objectworld*. The shadow of the line in the figures represents the standard error for each experimental trail. Notice that the EVDs for SIRL and DSIRL are both decreasing as the number and the trajectory length of expert demonstrations, and the portion size in the representative trajectory class are increasing. A notable point in Figure 3 is that very few expert demonstrations (less than 200) for our approach also yields a small EVDs, which manifests the merit of Monte Carlo mechanism in our approach.

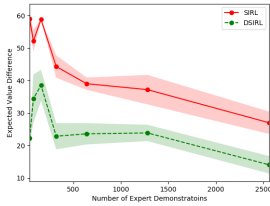


Figure 3: Results under 40, 80, 160, 320, 640, 1280 and 2560 expert demonstrations.

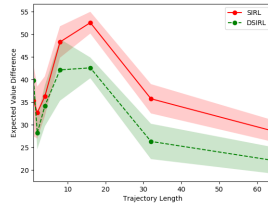


Figure 4: Results under 1, 2, 4, 8, 16, 32, and 64 grid size trajectory length of expert demonstrations.

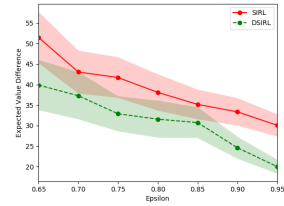


Figure 5: Results under 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, and 0.95 portion size in $\mathcal{C}_\epsilon^E$.

5 CONCLUSION

In this paper, we propose a generalized problem SIRL for the IRL problem to get the distribution of reward functions. The new problem is well-posed and we employ the method of MCEM to give the first *succinct, robust, and transferable* solution. In the experiment, we evaluate our approach on the *objectworld* and the experimental results confirm the effectiveness of our approach.

REFERENCES

- Pieter Abbeel and Andrew Y Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 1. ACM, 2004.
- Pieter Abbeel, Dmitri Dolgov, Andrew Y Ng, and Sebastian Thrun. Apprenticeship learning for motion planning with application to parking lot navigation. In *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1083–1090. IEEE, 2008.
- Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, (5): 834–846, 1983.
- James G Booth and James P Hobert. Maximizing generalized linear mixed model likelihoods with an automated monte carlo em algorithm. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 61(1):265–285, 1999.
- Brian S Caffo, Wolfgang Jank, and Galin L Jones. Ascent-based monte carlo expectation–maximization. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):235–251, 2005.
- KS Chan and Johannes Ledolter. Monte carlo em estimation for time series models involving counts. *Journal of the American Statistical Association*, 90(429):242–252, 1995.
- Jaedeug Choi and Kee-Eung Kim. Map inference for bayesian inverse reinforcement learning. In *Advances in Neural Information Processing Systems*, pp. 1989–1997, 2011.
- Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *arXiv preprint arXiv:1611.03852*, 2016a.
- Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning*, pp. 49–58, 2016b.
- Gersende Fort, Eric Moulines, et al. Convergence of the monte carlo expectation maximization for curved exponential families. *The Annals of Statistics*, 31(4):1220–1259, 2003.
- Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. *arXiv preprint arXiv:1710.11248*, 2017.
- Karol Hausman, Yevgen Chebotar, Stefan Schaal, Gaurav Sukhatme, and Joseph J Lim. Multi-modal imitation learning from unstructured demonstrations using generative adversarial nets. In *Advances in Neural Information Processing Systems*, pp. 1235–1245, 2017.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in neural information processing systems*, pp. 4565–4573, 2016.
- Jonathan Ho, Jayesh Gupta, and Stefano Ermon. Model-free imitation learning with policy optimization. In *International Conference on Machine Learning*, pp. 2760–2769, 2016.
- William Kruskal and Frederick Mosteller. Representative sampling, iii: The current statistical literature. *International Statistical Review/Revue Internationale de Statistique*, pp. 245–265, 1979.
- Sergey Levine, Zoran Popovic, and Vladlen Koltun. Nonlinear inverse reinforcement learning with gaussian processes. In *Advances in Neural Information Processing Systems*, pp. 19–27, 2011.
- Yunzhu Li, Jiaming Song, and Stefano Ermon. Infogail: Interpretable imitation learning from visual demonstrations. In *Advances in Neural Information Processing Systems*, pp. 3812–3822, 2017.
- Bernard Michini and Jonathan P How. Improving the efficiency of bayesian inverse reinforcement learning. In *2012 IEEE International Conference on Robotics and Automation*, pp. 3651–3656. IEEE, 2012.
- Andrew William Moore. Efficient memory-based learning for robot control. 1990.

- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pp. 561–577, 2018.
- Andrew Y Ng, Stuart J Russell, et al. Algorithms for inverse reinforcement learning. In *Icml*, volume 1, pp. 2, 2000.
- Deepak Ramachandran and Eyal Amir. Bayesian inverse reinforcement learning. In *IJCAI*, volume 7, pp. 2586–2591, 2007.
- Nathan D Ratliff, J Andrew Bagnell, and Martin A Zinkevich. Maximum margin planning. In *Proceedings of the 23rd international conference on Machine learning*, pp. 729–736. ACM, 2006.
- Stuart Russell. Learning agents for uncertain environments. In *Proceedings of the eleventh annual conference on Computational learning theory*, pp. 101–103, 1998.
- Umar Syed and Robert E Schapire. A game-theoretic approach to apprenticeship learning. In *Advances in neural information processing systems*, pp. 1449–1456, 2008.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.
- Ziyu Wang, Josh S Merel, Scott E Reed, Nando de Freitas, Gregory Wayne, and Nicolas Heess. Robust imitation of diverse behaviors. In *Advances in Neural Information Processing Systems*, pp. 5320–5329, 2017.
- Greg CG Wei and Martin A Tanner. A monte carlo implementation of the em algorithm and the poor man’s data augmentation algorithms. *Journal of the American statistical Association*, 85(411): 699–704, 1990.
- Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*, 2015.
- Brian D Ziebart. Modeling purposeful adaptive behavior with the principle of maximum causal entropy. 2010.
- Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pp. 1433–1438. Chicago, IL, USA, 2008.

A APPENDIX

The pseudocode of SIRL in Section 3 is as follows.

Algorithm 2: Stochastic Inverse Reinforcement Learning

Input: Model-based environment $(\mathcal{S}, \mathcal{A}, \mathcal{T})$ and expert demonstrations ζ^E , Monte Carlo sample size N_0 , and preset thresholds δ_{MCEM} and ϵ_{MCEM} .

Output: Reward conditional probability distribution $\mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta^*)$.

Initialization: Randomly initialization of profile parameter $\Theta^0 := (\Theta_1^0, \Theta_2^0)$;

while *stopping criteria not satisfied (refer to Section 3.2)* **do**

 Draw N_t reward weights $\mathcal{W}_i^{\Theta^t} \sim \mathcal{D}^{\mathcal{R}}(\mathcal{W}|\zeta^E, \Theta_2^t)$ to compose learning task $\mathcal{M}_i^t$ with uniformly drawn trajectory element set $\mathcal{O}_i^t$;

 # *First Stage: Monte Carlo estimation of weights for reward function;*

for $\mathcal{M}_i^t$ **do**

 Evaluate $\nabla_{(\Theta_1)_i} \log g_{\mathcal{M}_i^t}(\mathcal{O}_i^t|\mathcal{W}_i^{\Theta^t}, (\Theta_1)_i)$;

 Compute updated weight parameter

$\mathcal{W}_i^{m\Theta^t} \leftarrow \mathcal{W}_i^{\Theta^t} + \sum_{i=1}^m \lambda_i^t \cdot \nabla_{(\Theta_1)_i} \log g_{\mathcal{M}_i^t}(\mathcal{O}_i^t|\mathcal{W}_i^{\Theta^t}, (\Theta_1)_i)$;

end

 Update $\Theta_1^{t+1} \leftarrow \{\mathcal{W}_i^{m\Theta^t}\}_{i=1}^{N_t}$;

 # *Second Stage: Fit GMM with m -step reward weight $\{\mathcal{W}_i^{m\Theta^t}\}_{i=1}^{N_t}$ with EM parameter initialization Θ_2^t ;*

while *EM not converge* **do**

 Expectation Step: $\gamma_{ij} \leftarrow \frac{\alpha_j \cdot \mathcal{N}(\mathcal{W}_i^{m\Theta^t}|\mu_j, \Sigma_j)}{\sum_{k=1}^K \alpha_k \cdot \mathcal{N}(\mathcal{W}_i^{m\Theta^t}|\mu_k, \Sigma_k)}$;

 Maximization Step:

$$\begin{aligned} \mu_j &\leftarrow \frac{\sum_{i=1}^{N_t} \gamma_{ij} \cdot \mathcal{W}_i^{m\Theta^t}}{\sum_{i=1}^{N_t} \gamma_{ij}}; & \alpha_j &\leftarrow \frac{1}{N_t} \cdot \sum_{i=1}^{N_t} \gamma_{ij}; \\ \Sigma_j &\leftarrow \frac{\sum_{i=1}^{N_t} \gamma_{ij} \cdot (\mathcal{W}_i^{m\Theta^t} - \mu_j) \cdot (\mathcal{W}_i^{m\Theta^t} - \mu_j)^T}{\sum_{i=1}^{N_t} \gamma_{ij}}; \end{aligned}$$

end

 Update Θ_2^{t+1} and profile parameter $\Theta^{t+1} \leftarrow (\Theta_1^{t+1}, \Theta_2^{t+1})$;

end
